AWS VPC Peering

Private Connectivity Between Two VPCs

Project report:

Student	Shubham Badwane
Batch	Aws Internship Batch 1
Project	AWS VPC Peering: connect two VPCs and prove private connectivity
Region	Asia Pacific (Mumbai), ap-south-1
Completion date	25 October 2026

1. Objective

EWB has an application in VPC A and a second workload in VPC B. Both application servers must stay private. The goal of this project is to connect the two VPCs with VPC peering so the servers can exchange traffic using their private IPv4 addresses, with one public bastion host in VPC A used for administration.

This report demonstrates that:

- The two VPCs use non-overlapping CIDRs (10.10.0.0/16 and 10.20.0.0/16).
- Public and private subnets, route tables and security groups were created for the lab.
- Peering was activated and routes were added on both sides, so traffic can go out and come back.
- A ping failed before peering and succeeded in both directions after peering; App-B was reached by SSH through the bastion.
- The lab resources were then cleaned up.

2. Architecture

There is one internet entry point (the bastion in VPC A) and two private application servers. VPC B has no internet gateway. Administration goes laptop → bastion → App-A or App-B over SSH. Application traffic goes App-A ↔ App-B over private IPs through the peering connection.

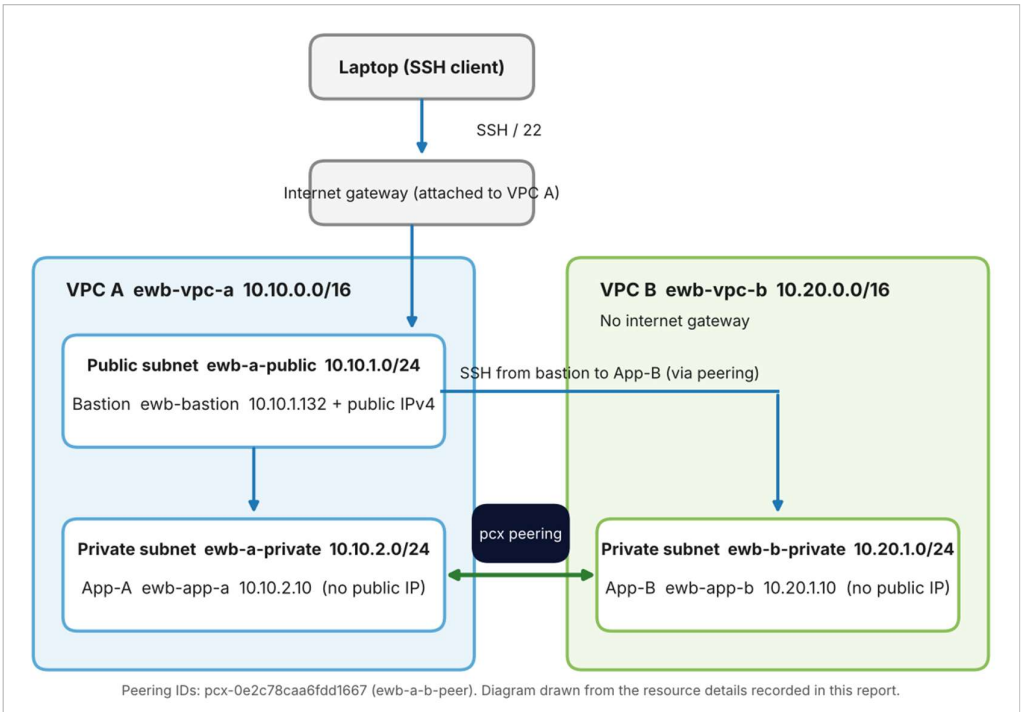


Figure 1. Architecture diagram of the lab (drawn from the resource details in this report; not a console screenshot).

Peering does not give VPC B internet access through VPC A, and it is one-to-one and non-transitive.

3. Resource Inventory

IDs below were read from the screenshots in this report. Items not visible in any screenshot are marked "Unavailable".

Resource	Name	ID	CIDR / address
VPC A	ewb-vpc-a	vpc-0c6e5be6e64e0d7bf	10.10.0.0/16
VPC B	ewb-vpc-b	vpc-0da9ec199dd6667a9	10.20.0.0/16
Public subnet (VPC A)	ewb-a-public	subnet-047ce357167336525	10.10.1.0/24, ap-south-1a
Private subnet (VPC A)	ewb-a-private	subnet-0471c5e336c0a7820	10.10.2.0/24, ap-south-1a
Private subnet (VPC B)	ewb-b-private	subnet-07c97069aeabdfe03	10.20.1.0/24, ap-south-1b
Internet gateway	ewb-igw-a (name not visible)	igw-06d61f92f5ce5d431	Attached to VPC A
Route table (public)	ewb-rt-a-public	rtb-0d982f67b99d9d4d1	VPC A
Route table (private A)	ewb-rt-a-private	rtb-0cf31594175bd7612	VPC A
Route table (private B)	ewb-rt-b-private	rtb-06c78fe06c13e81a7	VPC B
Main route table (VPC A)	Unnamed	rtb-0d897674b1ffa62be	VPC A
Security group	ewb-sg-bastion	sg-09757c8b641fbf85e	VPC A
Security group	ewb-sg-app-a	sg-0f2639cff8e33cd32	VPC A
Security group	ewb-sg-app-b	sg-07a7eed9c8336150a	VPC B
EC2 instance	ewb-bastion (t3.micro)	i-0d4f55bdbe2824c6d	10.10.1.132; public 13.127.253.208
EC2 instance	ewb-app-a (t3.micro)	i-0291086c9bfe653e3	10.10.2.10; no public IP
EC2 instance	ewb-app-b (t3.micro)	i-0d30d1cff365de028	10.20.1.10; no public IP
Peering connection	ewb-a-b-peer	pcx-0e2c78caa6fdd1667	Requester VPC A, accepter VPC B
Key pair	ewb-lab	Unavailable	Used from ~/.ssh/ewb-lab.pem

Note on the bastion IP: the workbook plan uses 10.10.1.10 for the bastion, but the console shows AWS assigned 10.10.1.132. The App-A and App-B terminals also show "last login from 10.10.1.132". The workbook says to replace the bastion's /32 security-group rules with the actual value in this case; the Source column of the security-group screenshots is not visible, so the exact source values are unavailable (see Section 6).

Tags (Project / Student): not visible in any screenshot, so unavailable.

4. VPCs and Subnets

VPC A (10.10.0.0/16) has a public subnet for the bastion and a private subnet for App-A. VPC B (10.20.0.0/16) has one private subnet for App-B. All subnets use /24 ranges inside their own VPC range, and none of the CIDRs overlap. Auto-assign public IPv4 is "No" on all subnets shown.

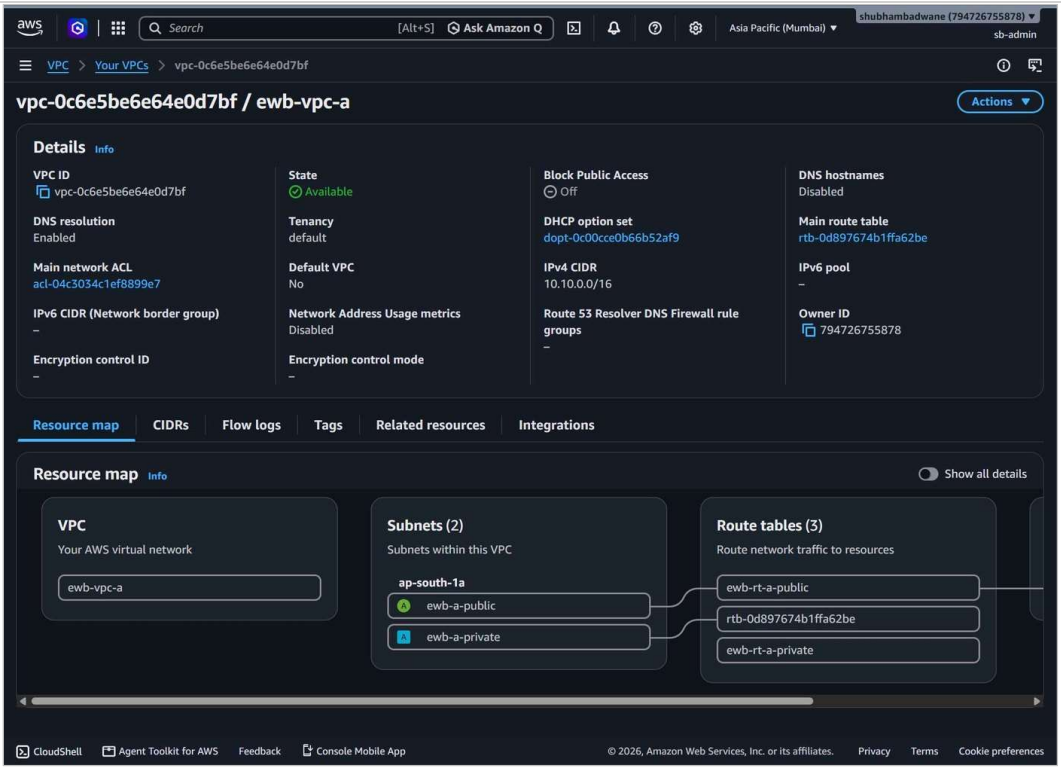


Figure 2. Console, VPC A (ewb-vpc-a): state Available, IPv4 CIDR 10.10.0.0/16; resource map shows 2 subnets and 3 route tables.

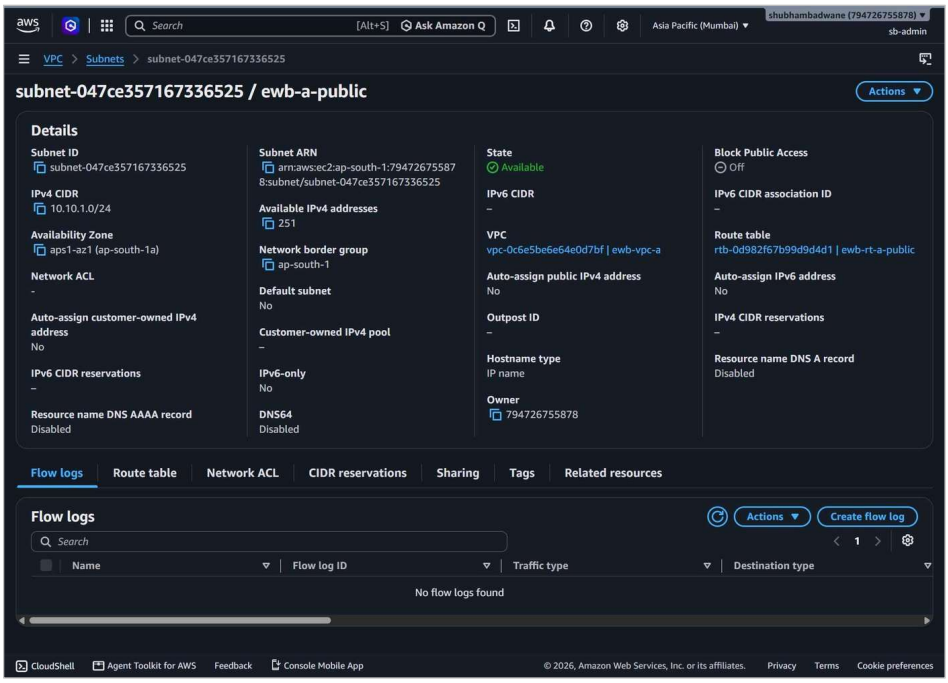


Figure 3. Console, subnet ewb-a-public: 10.10.1.0/24 in ap-south-1a, route table ewb-rt-a-public.

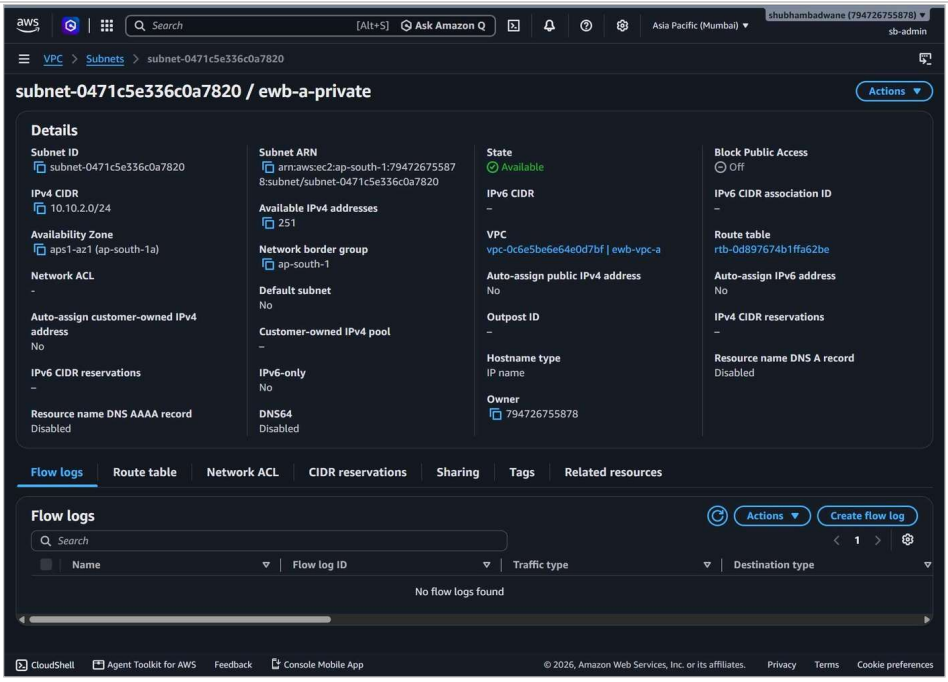


Figure 4. Console, subnet ewb-a-private: 10.10.2.0/24 in ap-south-1a. The route table shown is rtb-0d897674b1ffa62be (see note in Section 5).

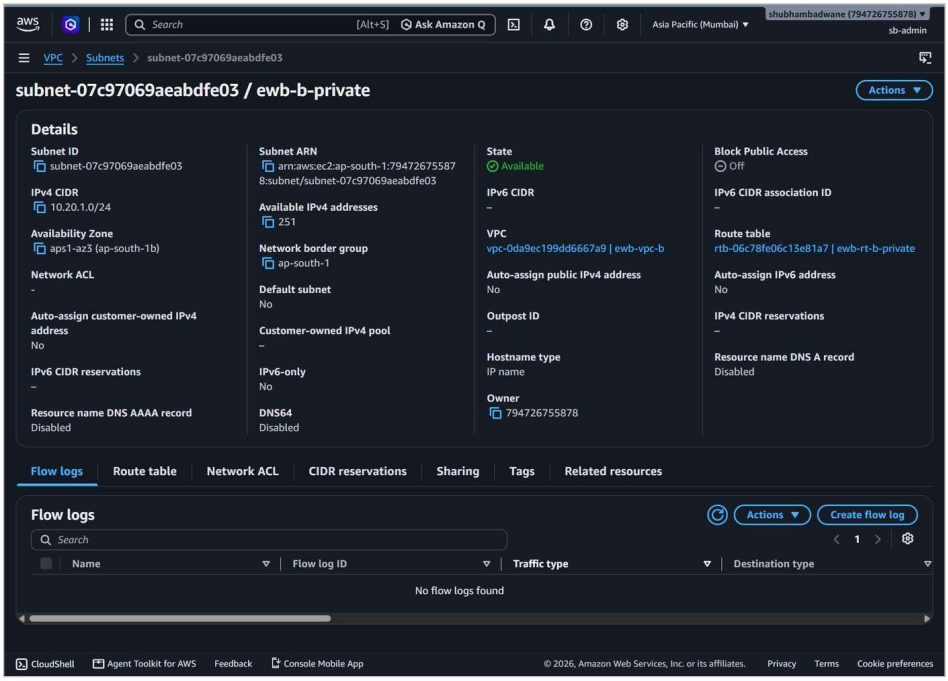


Figure 5. Console, subnet ewb-b-private in VPC B (ewb-vpc-b): 10.20.1.0/24, route table ewb-rt-b-private.

A VPC B details page was not captured; its ID and CIDR are taken from the subnet, peering and instance screenshots.

5. Route Tables

5.1 Before peering

The public route table sends internet traffic (0.0.0.0/0) to the internet gateway. The private route table had only its local VPC route.

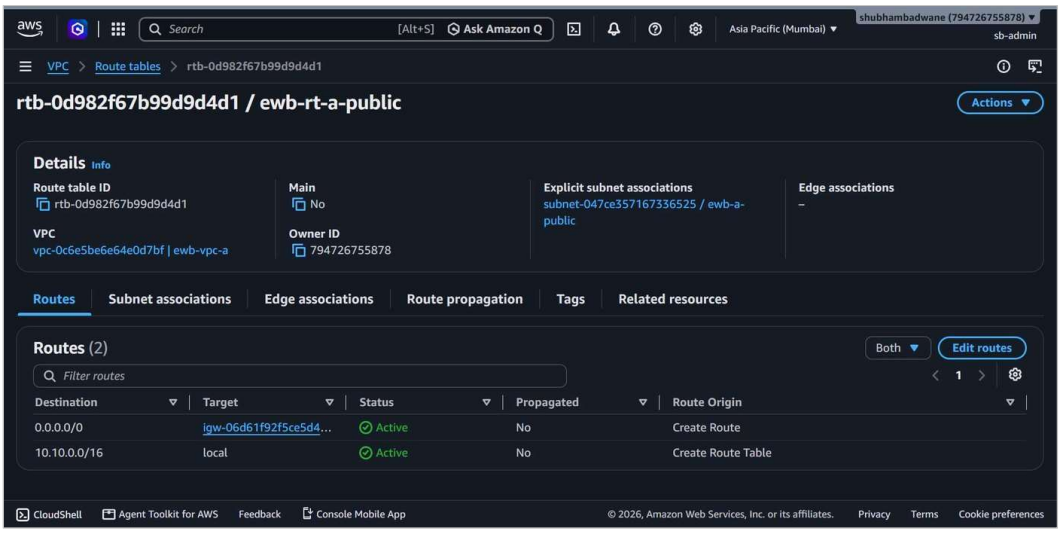


Figure 6. Console, ewb-rt-a-public before peering: 0.0.0.0/0 → igw-06d61f92f5ce5d431 and 10.10.0.0/16 → local; explicit association with ewb-a-public.

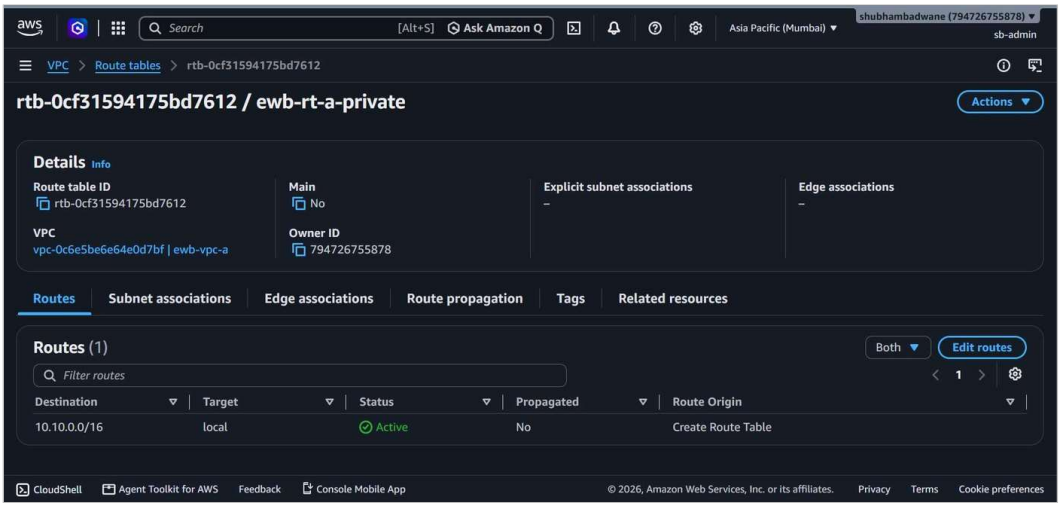


Figure 7. Console, ewb-rt-a-private before peering: only 10.10.0.0/16 → local.

5.2 After peering

After the peering connection was active, routes to the other VPC were added on both sides. These are the routes seen in the screenshots:

Route table	Destination	Target	Purpose
ewb-rt-a-private	10.10.0.0/16	local	Existing VPC A route
ewb-rt-a-private	10.20.0.0/16	pcx-0e2c78caa6fdd1667	App-A to VPC B (forward path)
ewb-rt-b-private	10.20.0.0/16	local	Existing VPC B route
ewb-rt-b-private	10.10.0.0/16	pcx-0e2c78caa6fdd1667	Return path to VPC A

Route table	Destination	Target	Purpose
ewb-rt-a-public	0.0.0.0/0	igw-06d61f92f5ce5d431	Bastion internet access
ewb-rt-a-public	10.10.0.0/16	local	Bastion to App-A
ewb-rt-a-public	10.20.0.0/16	pcx-0e2c78caa6fdd1667	Bastion SSH to App-B

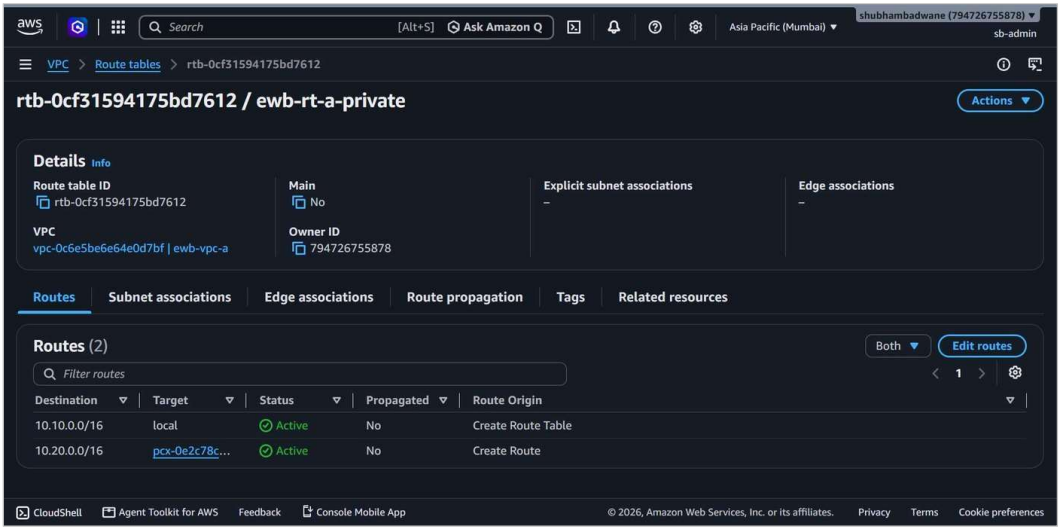


Figure 8. Console, ewb-rt-a-private after peering: 10.20.0.0/16 → pcx-0e2c78c… (Active).

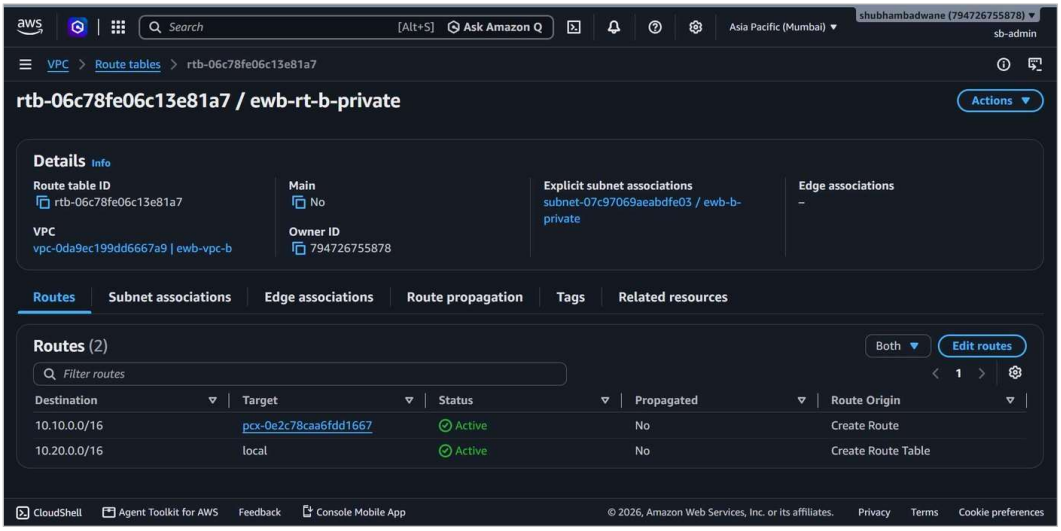


Figure 9. Console, ewb-rt-b-private after peering: 10.10.0.0/16 → pcx-0e2c78caa6fdd1667 (Active); associated with ewb-b-private.

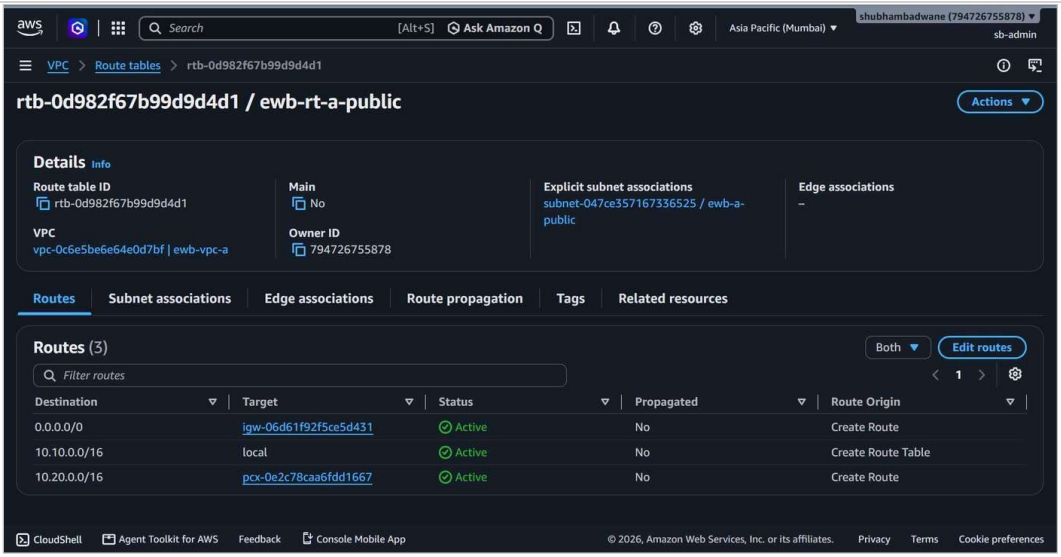


Figure 10. Console, ewb-rt-a-public after peering: three routes (igw, local, peering to 10.20.0.0/16).

Evidence note: the subnet screenshot (Figure 4) shows ewb-a-private using the VPC A main route table rtb-0d897674b1ffa62be, and both ewb-rt-a-private screenshots show no explicit subnet association. The routes of the main route table were not captured. The successful ping from App-A (Section 9) shows App-A could reach VPC B, but the screenshots do not show which route table carried that route.

6. Security Groups

Security groups control which traffic is allowed in. They are stateful, so replies to allowed traffic are allowed automatically. The workbook design is:

Security group	Inbound rule	Source (workbook design)
ewb-sg-bastion	SSH / TCP 22	Student's current public IPv4 /32
ewb-sg-app-a	SSH / TCP 22	Bastion private IP /32
ewb-sg-app-a	Custom ICMP IPv4, Echo Request	App-B 10.20.1.10/32
ewb-sg-app-b	SSH / TCP 22	Bastion private IP /32
ewb-sg-app-b	Custom ICMP IPv4, Echo Request	App-A 10.10.2.10/32

The screenshots confirm the rule types and ports below. The Source column is cut off in all three screenshots, so the actual source values are **unavailable**.

- **ewb-sg-bastion** (sg-09757c8b641fbf85e, VPC A): 1 inbound rule, SSH TCP 22.
- **ewb-sg-app-a** (sg-0f2639cff8e33cd32, VPC A): 2 inbound rules, Custom ICMP IPv4 Echo Request and SSH TCP 22.
- **ewb-sg-app-b** (sg-07a7eed9c8336150a, VPC B): 2 inbound rules, SSH TCP 22 and Custom ICMP IPv4 Echo Request.

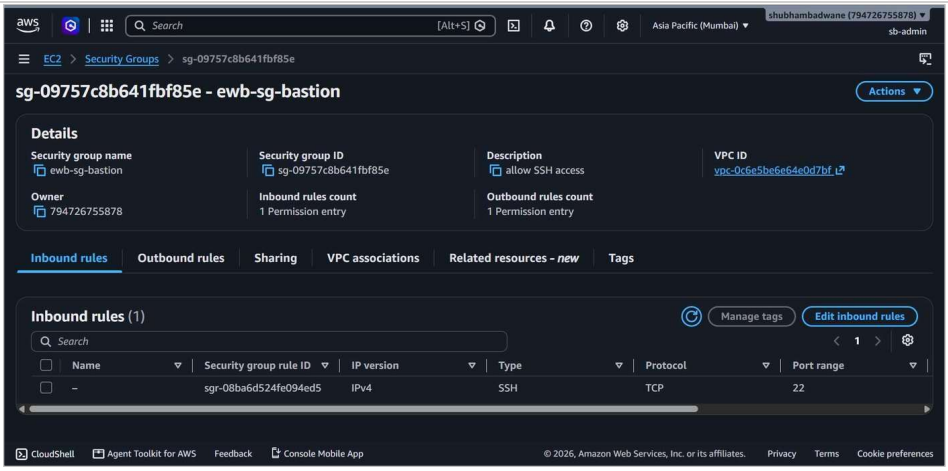


Figure 11. Console, ewb-sg-bastion: inbound rule SSH / TCP / 22 (source not visible).

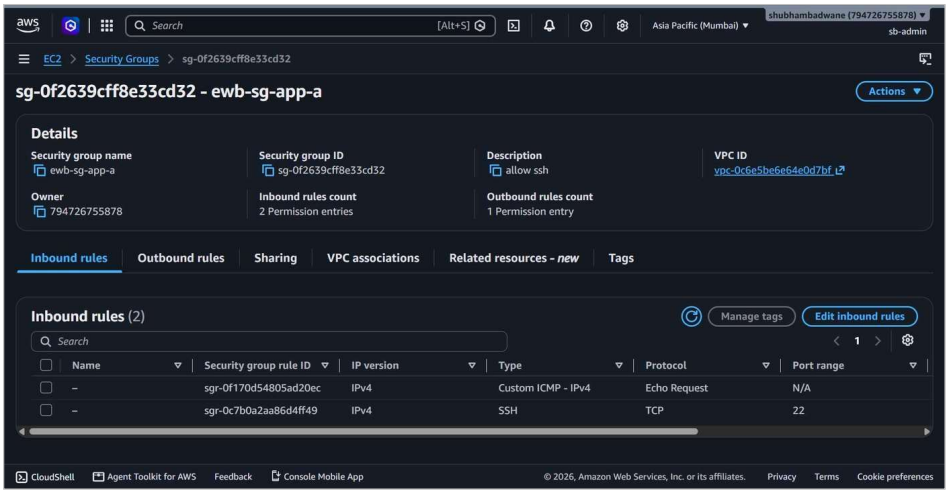


Figure 12. Console, ewb-sg-app-a: inbound rules ICMP Echo Request and SSH / TCP / 22 (sources not visible).

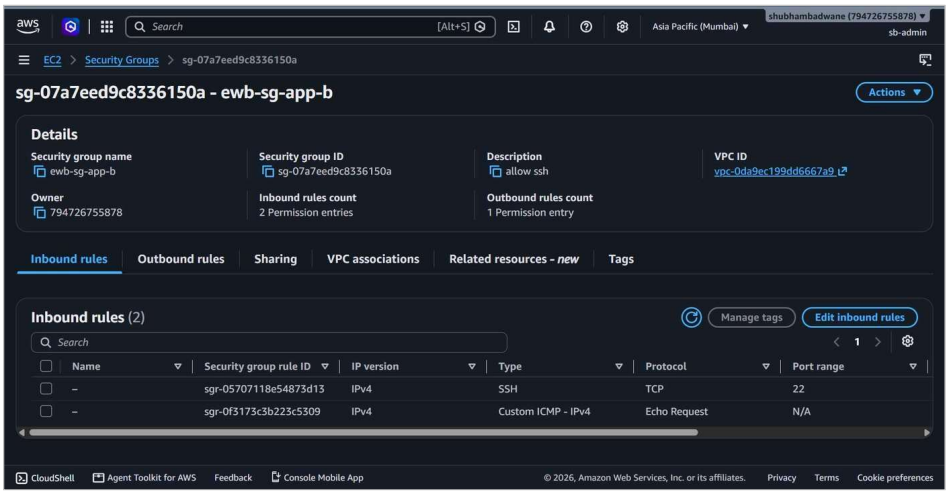


Figure 13. Console, ewb-sg-app-b: inbound rules SSH / TCP / 22 and ICMP Echo Request (sources not visible).

7. EC2 Instances

Three t3.micro instances were launched with Amazon Linux 2023. Only the bastion has a public IPv4 address; App-A and App-B have none, which proves they are private.

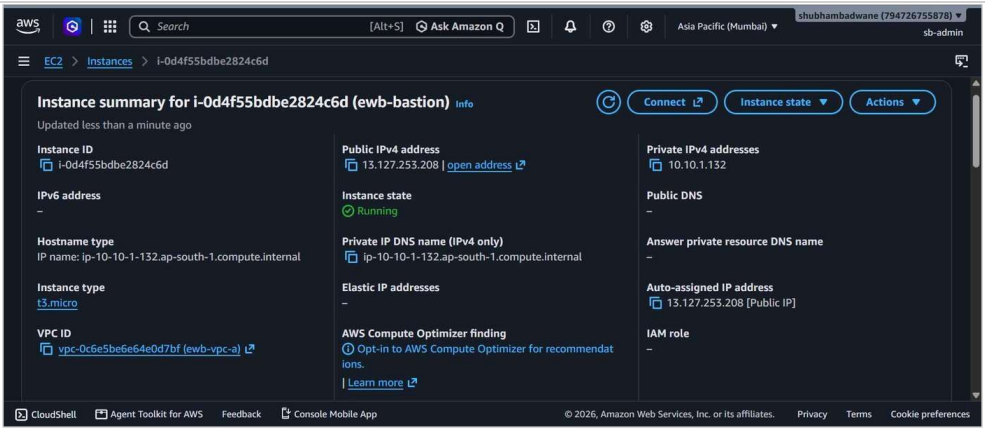


Figure 14. Console, ewb-bastion (i-0d4f55bdbe2824c6d): Running, public IPv4 13.127.253.208, private IPv4 10.10.1.132, VPC ewb-vpc-a.

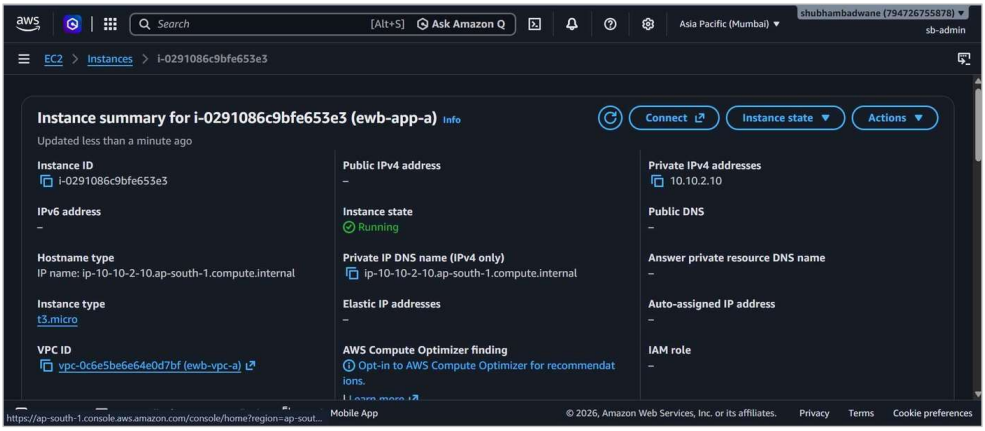


Figure 15. Console, ewb-app-a (i-0291086c9bfe653e3): Running, private IPv4 10.10.2.10, Public IPv4 address empty (–).

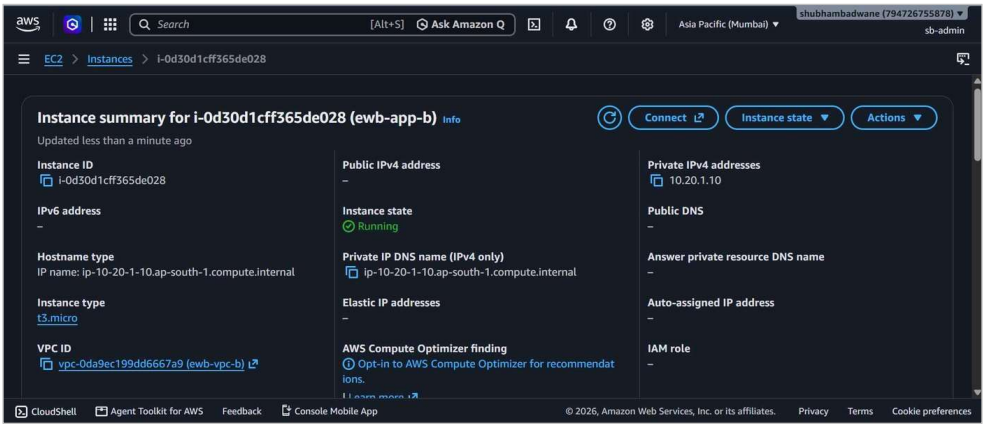


Figure 16. Console, ewb-app-b (i-0d30d1cff365de028): Running, private IPv4 10.20.1.10, Public IPv4 address empty (–), VPC ewb-vpc-b.

8. VPC Peering Setup

The peering connection ewb-a-b-peer was requested from VPC A (requester) to VPC B (accepter) in the same account and Region, then accepted. The request must be accepted even when both VPCs are in the same account. Peering alone does not carry traffic; the routes in Section 5.2 and the security-group rules are also needed.

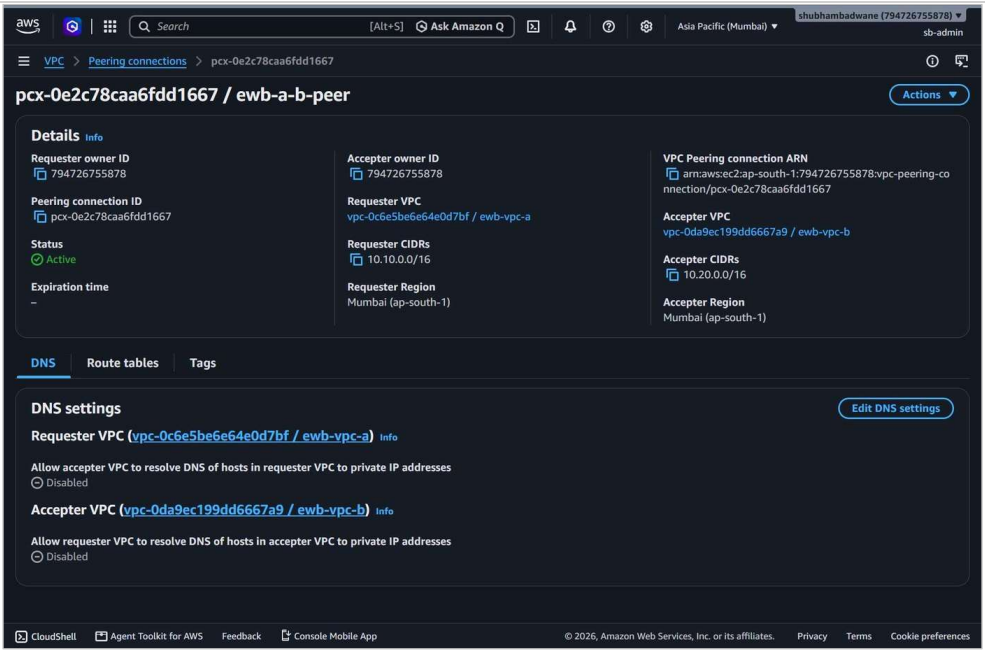


Figure 17. Console, peering connection pcx-0e2c78caa6fdd1667 (ewb-a-b-peer): Status Active; requester vpc-0c6e5be6e64e0d7bf (10.10.0.0/16), accepter vpc-0da9ec199dd6667a9 (10.20.0.0/16), both in Mumbai (ap-south-1).

9. Traffic Paths and Test Results

9.1 Forward and return path in simple language

Forward path (App-A → App-B). App-A (10.10.2.10) sends a ping to 10.20.1.10. That address is not inside VPC A, so the route table of App-A's subnet looks for a matching route and finds 10.20.0.0/16 → peering connection. The packet crosses the peering link into VPC B. App-B's security group allows the ICMP Echo Request from App-A, so App-B receives it.

Return path (App-B → App-A). App-B replies to 10.10.2.10. VPC B's route table must also know where 10.10.0.0/16 is, and it does: 10.10.0.0/16 → peering connection. The reply travels back over the same link. Because security groups are stateful, the reply is allowed automatically. Without the return route the request would arrive but the reply would be lost, so ping would still fail.

SSH to App-B through the bastion. The laptop connects to the bastion's public IP. The bastion then opens a separate SSH connection to App-B's private IP. For that to work, the bastion's public subnet needs the 10.20.0.0/16 → peering route (it is more specific than 0.0.0.0/0, so it is chosen), and App-B must allow SSH from the bastion's private IP.

9.2 Test summary

#	Test	Where / command	Result (from screenshots)	Figure
1	SSH to bastion	Laptop (WSL): ssh -i ~/.ssh/ewb-lab.pem ec2-user@13.127.253.208	Connected; hostname ip-10-10-1-132	18
2	SSH to App-A via bastion	Laptop: ssh with ProxyCommand through the bastion to 10.10.2.10	Connected; hostname ip-10-10-2-10, IP 10.10.2.10	19
3	Baseline (before peering)	App-A: ping -c 4 10.20.1.10	4 sent, 0 received, 100% packet loss	20
4	Forward path (after routes)	App-A: ping -c 4 10.20.1.10	4 sent, 4 received, 0% loss; avg 0.517 ms	21

#	Test	Where / command	Result (from screenshots)	Figure
5	Reverse path (after routes)	App-B: ping -c 4 10.10.2.10	4 sent, 4 received, 0% loss; avg 0.514 ms	22
6	Private SSH to App-B	Laptop: ssh through the bastion to 10.20.1.10	Shell on ip-10-20-1-10 (Last login from 10.10.1.132)	23
7	Isolation	EC2 instance details of App-A and App-B	Public IPv4 address empty on both	15, 16
8	Controlled failure and recovery	Remove, then restore, App-B ICMP rule	Not performed: no evidence exists	None

The SSH connections used the same key and the bastion as a jump host through an explicit ProxyCommand (ssh -W), which does the same job as the ProxyJump entry in the workbook's ~/.ssh/config. No private key is included in this report.

9.3 Evidence: administration access

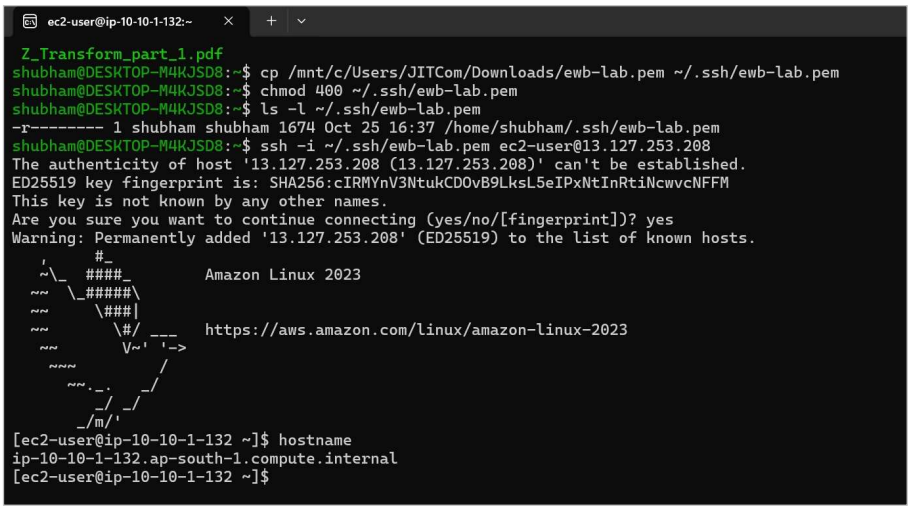


Figure 18. Laptop terminal (WSL), SSH to the bastion 13.127.253.208: connected as ec2-user; hostname ip-10-10-1-132.ap-south-1.compute.internal.

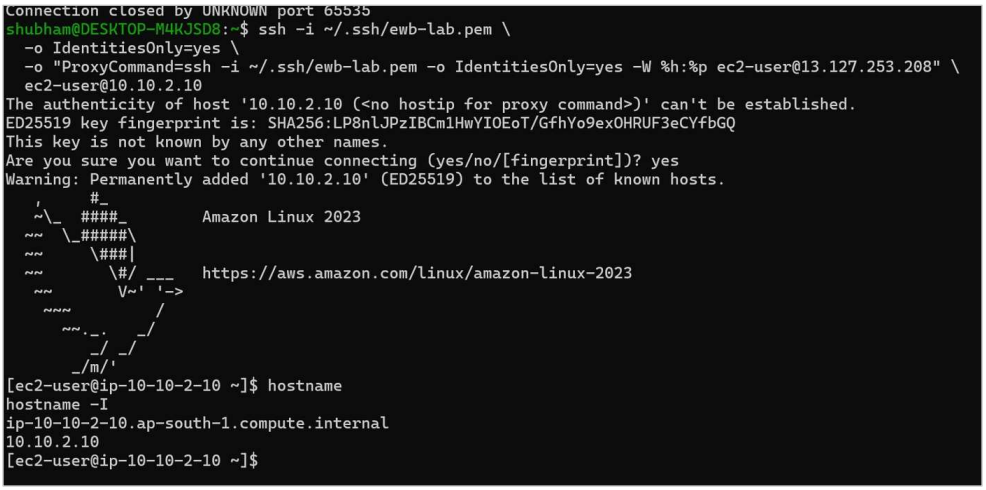


Figure 19. Laptop terminal, SSH to App-A 10.10.2.10 through the bastion: connected; hostname ip-10-10-2-10, hostname -I shows 10.10.2.10.

9.4 Evidence: baseline failure (before peering)

```
[ec2-user@ip-10-10-2-10 ~]$ hostname
hostname -I
ip-10-10-2-10.ap-south-1.compute.internal
10.10.2.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3095ms

[ec2-user@ip-10-10-2-10 ~]$
```

Figure 20. App-A terminal, ping -c 4 10.20.1.10 before peering: 4 packets transmitted, 0 received, 100% packet loss.

9.5 Evidence: forward and reverse ping (after peering)

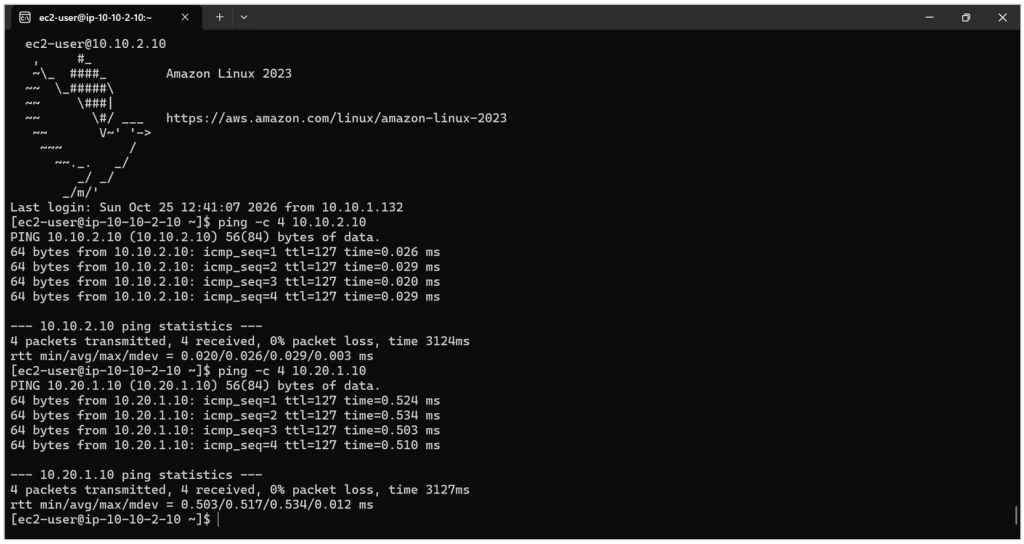


Figure 21. App-A terminal: the first ping is to App-A's own address (10.10.2.10); the forward test is ping -c 4 10.20.1.10: 4 transmitted, 4 received, 0% packet loss, rtt avg 0.517 ms.

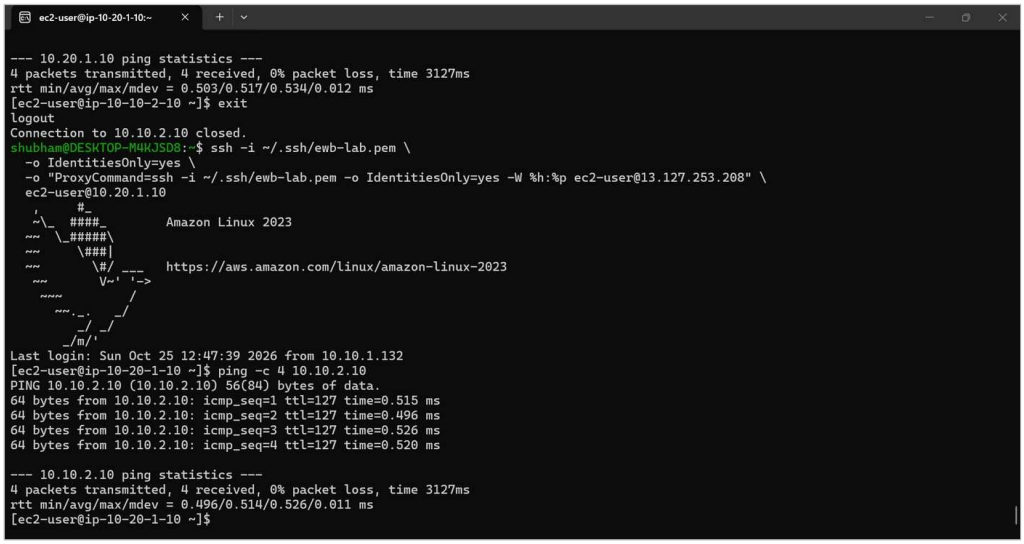


Figure 22. App-B terminal (ip-10-20-1-10): ping -c 4 10.10.2.10: 4 transmitted, 4 received, 0% packet loss, rtt avg 0.514 ms (reverse path).

9.6 Evidence: SSH to App-B through the bastion

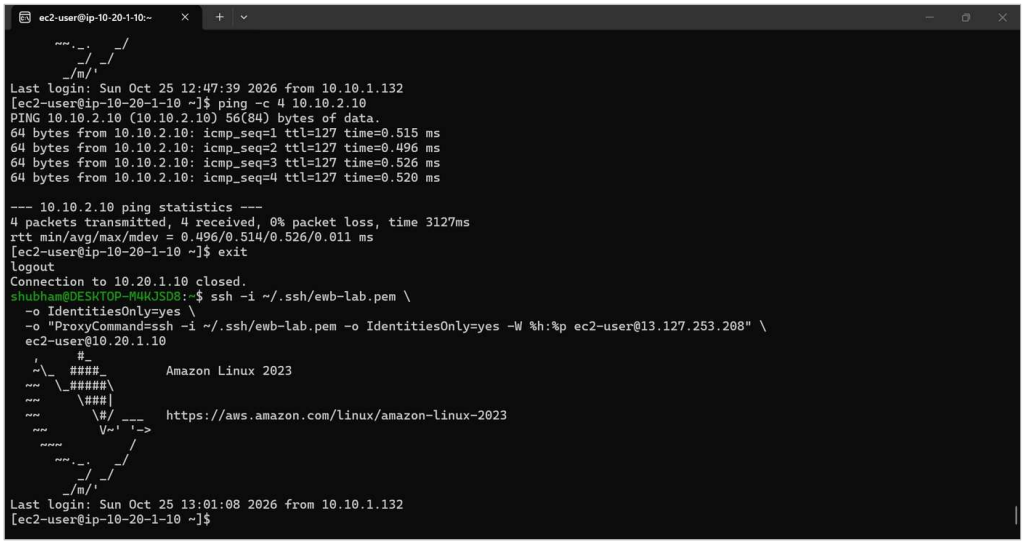


Figure 23. Laptop terminal, SSH to App-B 10.20.1.10 through the bastion: shell prompt [ec2-user@ip-10-20-1-10 ~]$ (private SSH works after peering).

9.7 Controlled failure and recovery

Ping depends on both the peering routes and the ICMP inbound rule on the destination security group. With routes and peering already Active, the ping from App-A to App-B returned 100% packet loss while ewb-sg-app-b had no ICMP rule for 10.10.2.41/32 (Figure 21). After the rule was added, the same command returned 4 of 4 replies (Figure 22). The reverse test from App-B showed the same pattern (Figure 20). This shows that routing and the peering connection alone do not guarantee reachability; the security-group rule must also allow the traffic.



Figure 21: Failure: ping App-A to App-B, 4 transmitted, 0 received, 100% packet loss

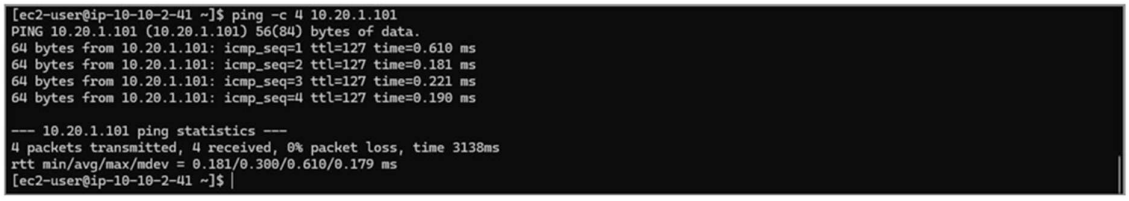


Figure 22: Recovery: ping App-A to App-B, 4 transmitted, 4 received, 0% packet loss after the ICMP rule was added

10. Cleanup

The lab resources were removed after testing. The table lists what the screenshots show.

Cleanup step	What the screenshot shows	Figure
Terminate instances	Banner: termination initiated for i-0d30d1cff365de028, i-0d4f55bdbe2824c6d, i-0291086c9bfe653e3. ewb-app-a and ewb-bastion show Terminated (ewb-app-b row is not visible).	24
Delete peering	pcx-0e2c78caa6fdd1667 deleted; “3 of 3 related routes were deleted”.	25
Delete route tables	ewb-rt-a-private (rtb-0cf31594175bd7612) and ewb-rt-a-public (rtb-0d982f67b99d9d4d1) deleted; ewb-rt-b-private still listed at that moment.	26

Cleanup step	What the screenshot shows	Figure
Delete security groups	Banner: “Successfully deleted 3 security groups”. The list below it still shows ewb-sg-bastion (list not refreshed in the capture).	27
Delete subnets	subnet-047ce357167336525, subnet-07c97069aeabdfe03, subnet-0471c5e336c0a7820 deleted.	28
Delete VPCs	Banner: “deleted vpc-0da9ec199dd6667a9 / ewb-vpc-b and 1 other resources”. The other resource is not named.	29
Internet gateway, key pair, EBS volumes, Elastic IP, billing check	No screenshot. Unavailable. (The bastion page shows no Elastic IP.)	None

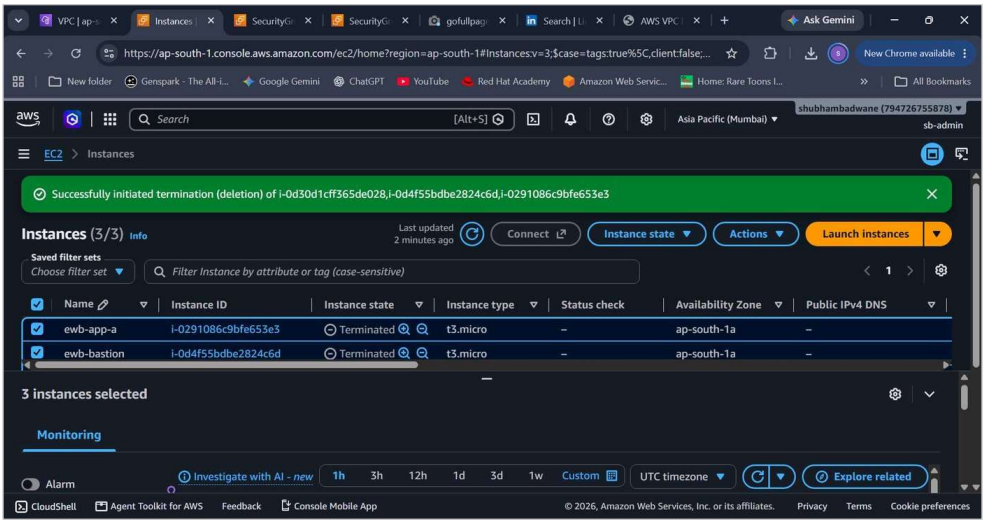


Figure 24. Console, EC2 Instances: termination of all three instances initiated; ewb-app-a and ewb-bastion show Terminated.

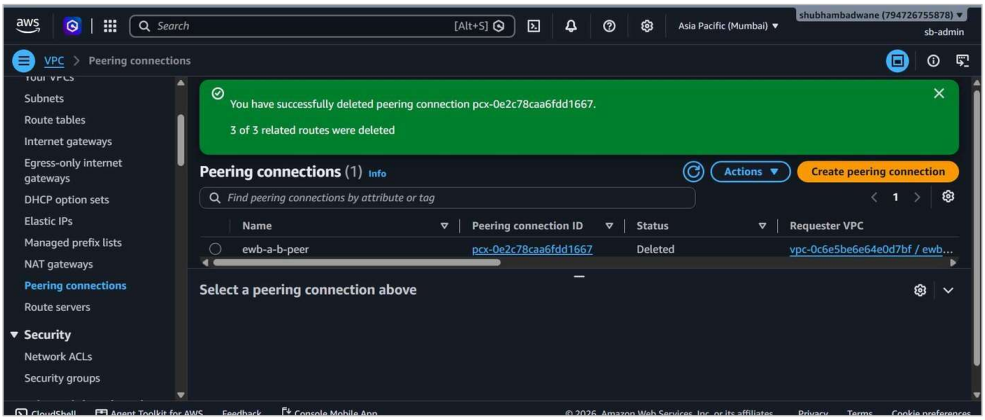


Figure 25. Console, VPC Peering connections: pcx-0e2c78caa6fdd1667 (ewb-a-b-peer) deleted, 3 of 3 related routes deleted.

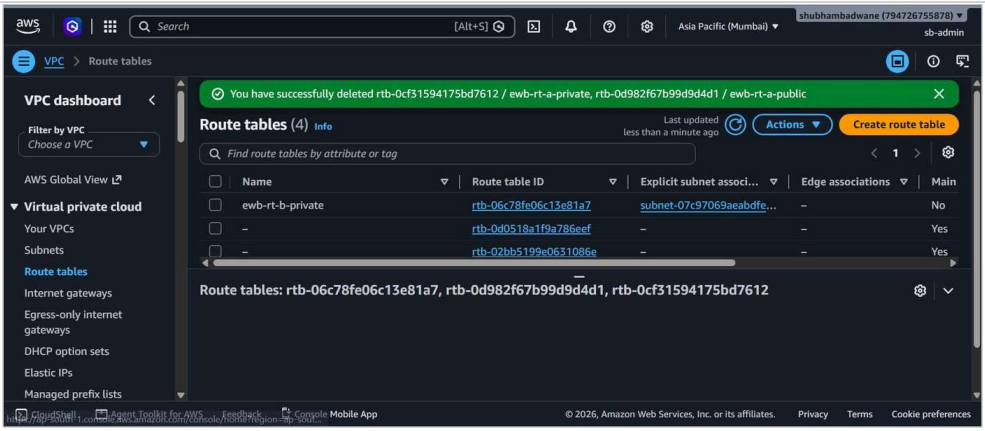


Figure 26. Console, Route tables: ewb-rt-a-private and ewb-rt-a-public deleted.

11. Conclusion

A private connection between VPC A and VPC B was built with VPC peering. The baseline ping failed before peering, and after the peering was activated and routes were added on both sides, ping worked in both directions with 0% packet loss. App-B, which has no public IP, was reached by SSH only through the bastion in VPC A. The main lessons are that peering is only the link: it needs routes on both sides and matching security-group rules, and peering does not give VPC B internet access or connect other VPCs.